

Experiment-4

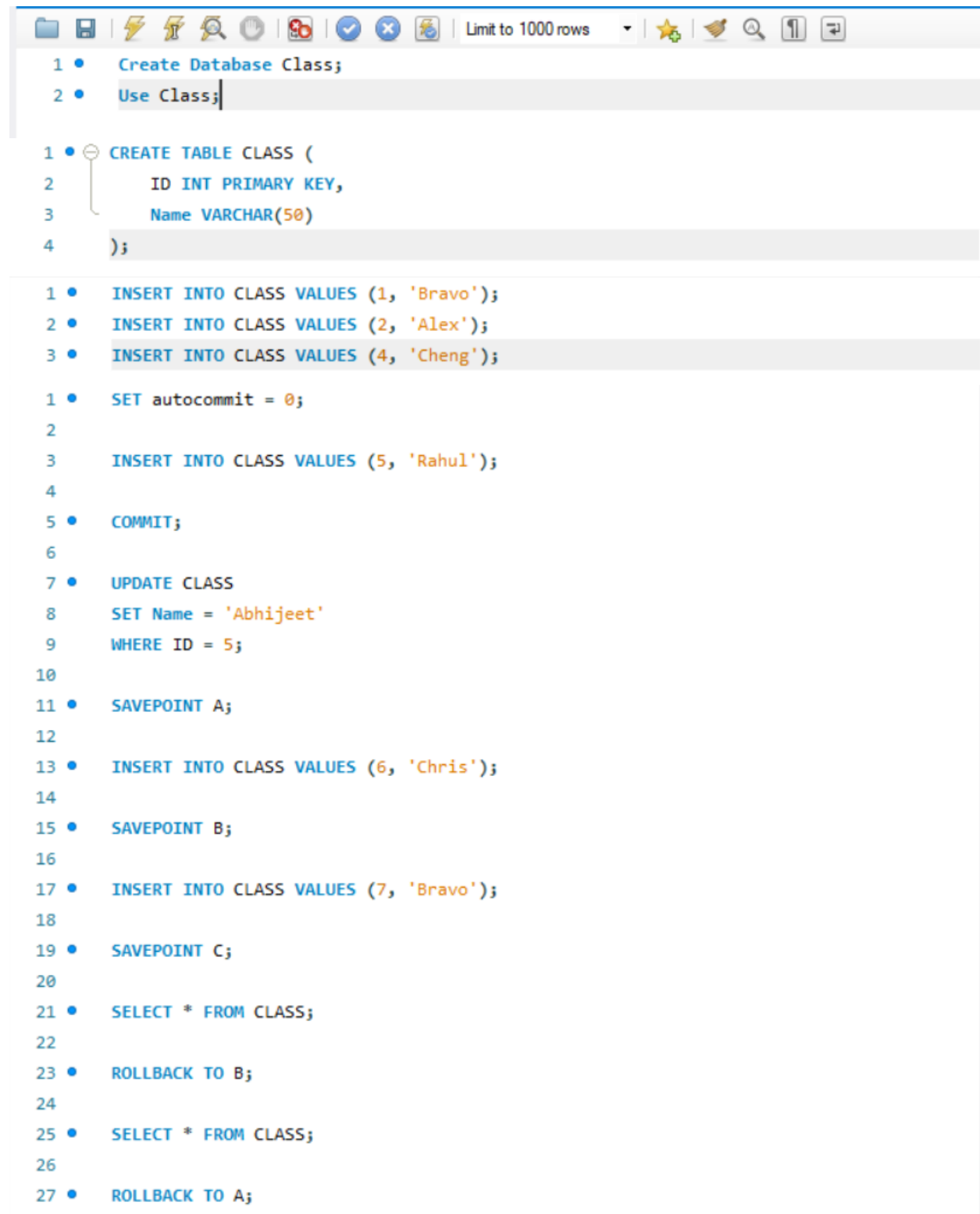
Aim: Consider the following table: Table: CLASS

ID	Name
1	Bravo
2	Alex
4	Cheng

Give the output of the following SQL script:

- **INSERT INTO class VALUES (5,'Rahul');**
- **COMMIT;**
- **UPDATE class SET name = 'Abhijeet' WHERE id= '5';**
- **SAVEPOINT A;**
- **INSERT INTO class VALUES (6, 'Chris');**
- **SAVEPOINT B;**
- **INSERT INTO class VALUES (7, 'Bravo');**
- **SAVEPOINT C**
- **SELECT * FROM class;**
- **ROLLBACK TO B;**
- **SELECT * FROM class;**
- **ROLLBACK TO A;**

Code:



```
1 • Create Database Class;
2 • Use Class;

1 • CREATE TABLE CLASS (
2     ID INT PRIMARY KEY,
3     Name VARCHAR(50)
4 );

1 • INSERT INTO CLASS VALUES (1, 'Bravo');
2 • INSERT INTO CLASS VALUES (2, 'Alex');
3 • INSERT INTO CLASS VALUES (4, 'Cheng');

1 • SET autocommit = 0;
2
3     INSERT INTO CLASS VALUES (5, 'Rahul');
4
5 • COMMIT;
6
7 • UPDATE CLASS
8     SET Name = 'Abhijeet'
9     WHERE ID = 5;
10
11 • SAVEPOINT A;
12
13 • INSERT INTO CLASS VALUES (6, 'Chris');
14
15 • SAVEPOINT B;
16
17 • INSERT INTO CLASS VALUES (7, 'Bravo');
18
19 • SAVEPOINT C;
20
21 • SELECT * FROM CLASS;
22
23 • ROLLBACK TO B;
24
25 • SELECT * FROM CLASS;
26
27 • ROLLBACK TO A;
```

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content

	ID	Name
▶	1	Bravo
	2	Alex
	4	Cheng
	5	Abhijeet
	6	Chris
	7	Bravo
★	NULL	NULL

Result Grid

Form Editor

	ID	Name
▶	1	Bravo
	2	Alex
	4	Cheng
	5	Abhijeet
	6	Chris
★	NULL	NULL

Experiment-4.2

Aim: Consider the following table: Table: ACCOUNT_TXN

Acc_ID Holder_Name Balance

101 Aman 5000

102 Bhavna 7000

103 Chirag 6000

Perform the following TCL-based SQL script and write the output after each SELECT statement:

INSERT INTO ACCOUNT_TXN VALUES (104, 'Diya', 4000);

SAVEPOINT S1;

UPDATE ACCOUNT_TXN SET Balance = Balance + 1500 WHERE Acc_ID = 101;

UPDATE ACCOUNT_TXN SET Balance = Balance - 1000 WHERE Acc_ID = 102;

SAVEPOINT S2;

DELETE FROM ACCOUNT_TXN WHERE Acc_ID = 103;

INSERT INTO ACCOUNT_TXN VALUES (105, 'Eshan', 3000);

SAVEPOINT S3;

UPDATE ACCOUNT_TXN SET Holder_Name = 'Bhavna Sharma' WHERE Acc_ID = 102;

SELECT * FROM ACCOUNT_TXN;

ROLLBACK TO S2;

SELECT * FROM ACCOUNT_TXN;

INSERT INTO ACCOUNT_TXN VALUES (106, 'Farah', 4500);

SAVEPOINT S4;

UPDATE ACCOUNT_TXN SET Balance = Balance + 500 WHERE Acc_ID IN (101, 104);

SELECT * FROM ACCOUNT_TXN;

ROLLBACK TO S4;

SELECT * FROM ACCOUNT_TXN;

COMMIT;

Student Tasks

Write the output of all 4 **SELECT** statements in tabular format. Identify which rows are permanently saved after **COMMIT**.

Mention which operations were undone by:

ROLLBACK TO S2

ROLLBACK TO S4

Write the final state of the table after **COMMIT**.

Additional Challenge (Conceptual)

```
1 • SET autocommit = 0;
2
3 INSERT INTO ACCOUNT_TXN VALUES (104, 'Diya', 4000);
4
5 • SAVEPOINT S1;
6
7 • UPDATE ACCOUNT_TXN
8 SET Balance = Balance + 1500
9 WHERE Acc_ID = 101;
10
11 • UPDATE ACCOUNT_TXN
12 SET Balance = Balance - 1000
13 WHERE Acc_ID = 102;
14
15 • SAVEPOINT S2;
16
17 • DELETE FROM ACCOUNT_TXN
18 WHERE Acc_ID = 103;
19
20 • INSERT INTO ACCOUNT_TXN VALUES (105, 'Eshan', 3000);
21 • SAVEPOINT S3;
22 • UPDATE ACCOUNT_TXN
23 SET Holder_Name = 'Bhavna Sharma'
24 WHERE Acc_ID = 102;
25 • SELECT * FROM ACCOUNT_TXN;
26 • ROLLBACK TO S2;
27 • SELECT * FROM ACCOUNT_TXN;
28 • INSERT INTO ACCOUNT_TXN VALUES (106, 'Farah', 4500);
29 • SAVEPOINT S4;
30 • UPDATE ACCOUNT_TXN
31 SET Balance = Balance + 500
32 WHERE Acc_ID IN (101,104);
33 • SELECT * FROM ACCOUNT_TXN;
34 • ROLLBACK TO S4;
35 • SELECT * FROM ACCOUNT_TXN;
36 • COMMIT;
```

Result Grid			
Filter Rows:			
Acc_ID	Holder_Name	Balance	
101	Aman	6500	
102	Bhavna	6000	
103	Chirag	6000	
104	Diya	4000	
106	Farah	4500	
* NULL	NULL	NULL	

Answer the following:

Ques:What is the difference between ROLLBACK and ROLLBACK TO SAVEPOINT?

Ans: ROLLBACK

- *Undoes all changes made in the current transaction*
- *Returns the database to the last committed state*
- *Does not require any savepoint*
- *Affects the entire transaction*
- *Simple and complete undo operation*

ROLLBACK TO SAVEPOINT

- *Undoes changes only up to a specific savepoint*
- *Does not cancel the entire transaction*
- *Requires a previously created SAVEPOINT*
- *Allows partial undo of changes*
- *Useful when working with large transactions*

Ques: Can we rollback to S3 after ROLLBACK TO S2? Why/Why not?

Ans: No, we cannot rollback to S3 after performing ROLLBACK TO S2

Reason:

- ***When we execute ROLLBACK TO S2, all savepoints created after S2 are removed***
- ***Since S3 was created after S2, it gets deleted***
- ***Therefore, S3 no longer exists in the transaction***

Ques: *What happens if COMMIT is executed before creating any savepoint?*

Ans:

- ***When COMMIT is executed, all changes made in the transaction are permanently saved in the database***
- ***It ends the current transaction***
- ***Since no SAVEPOINT was created, there is no intermediate point to return to***
- ***After COMMIT, you cannot use ROLLBACK or ROLLBACK TO SAVEPOINT for those changes***
- ***Any new operations will start a new transaction***